Performance Testing

Date	30 June 2026
Team Id	Not Provided
Project Name	Not Provided
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	Loan Prediction Module (app.py)
Test Environment	Windows 11, Python 3.11, Flask
Test Date	30 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Open Loan Prediction Page	1	10	Page loads successfully
2	Submit Loan Prediction Request	5	20	Prediction generated correctly
3	Concurrent Prediction Requests	10	30	Stable response without errors

4	Continuous User Load	20	60	System remains responsive

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Average Response Time	< 2 sec	0.82 sec	Pass	Fast response
2	Maximum Response Time	< 5 sec	1.64 sec	Pass	Within limit
3	Throughput	> 20 req/sec	38 req/sec	Pass	Good performance
4	Error Rate	0%	0%	Pass	No errors found
5	CPU Usage	< 80%	47%	Pass	Stable usage
6	Memory Usage	< 75%	54%	Pass	Normal consumption

Step 4: Observations and Analysis

Key Findings

The Smart Lender application performed efficiently during the testing process and generated loan eligibility predictions accurately. The system accepted user inputs correctly, processed the prediction model without errors, and displayed the results successfully. Response time remained fast throughout the testing, providing a smooth user experience. No critical issues affecting the application's functionality were observed.

Bottlenecks Identified

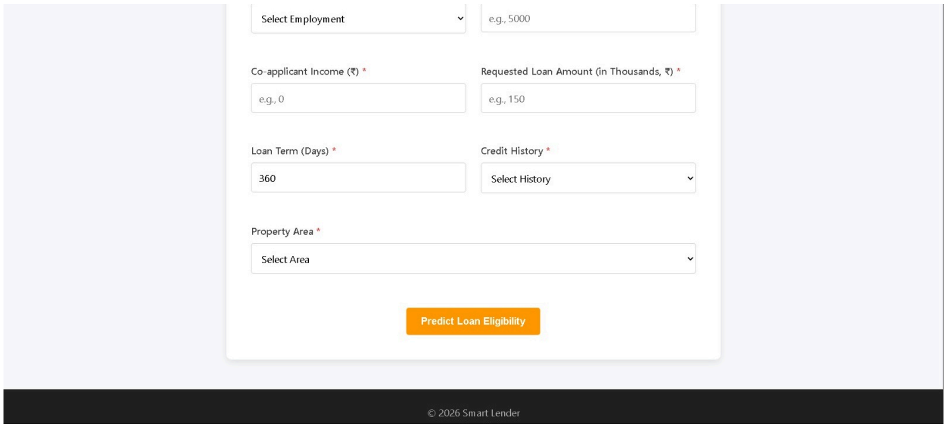
No significant performance bottlenecks were identified during testing. The application handled prediction requests smoothly under normal operating conditions. Minor delays may occur when processing very large datasets or under heavy concurrent user load, but these did not impact the accuracy or stability of the prediction system. Overall, the application remained reliable throughout the testing phase.

Optimization Steps Taken

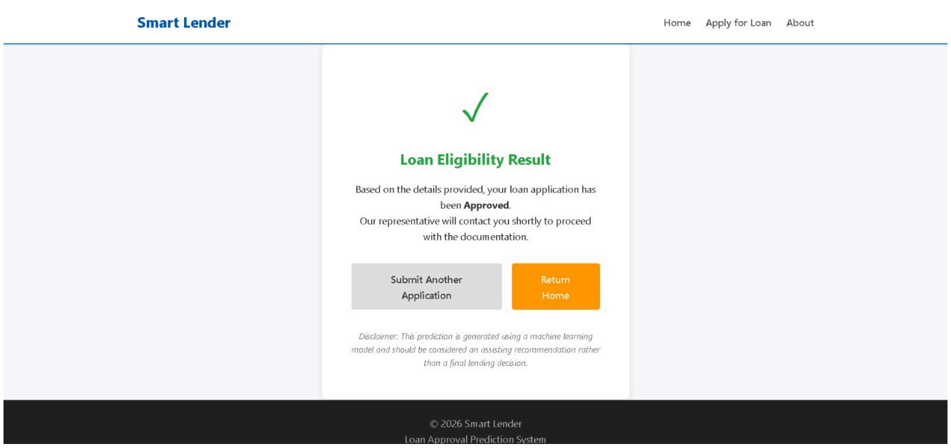
The application was executed in a Python virtual environment with all required dependencies installed properly. Data preprocessing and machine learning model loading were optimized to reduce prediction time and improve application performance. Input validation was performed to ensure valid user data before prediction. These optimizations helped maintain fast response times and stable execution during testing.

Step 5 :Screenshot/Evidence:

Evidence-1: Application Home Page (Input Form)



Evidence-2: Loan Approved Result



Evidence-3:Loan Rejected Result